
GACHAGO 直播弹幕抽奖

实习报告

实习内容

使用 HTML / CSS / 原生 JavaScript (ES Modules)
完成 B 站直播间弹幕抽奖网页

核心链路

登录 → 加房间 → 听弹幕 → 关键词入池 → 抽奖出名单

2026 年 10 月

目录

1 实习概述	3
2 整体框架搭建	3
2.1 总体架构	3
2.2 前端模块划分	3
2.3 运行方式	4
2.4 HTML 与 CSS 的基本构成	4
2.4.1 HTML 的基本构成	4
2.4.2 CSS 的基本构成	5
3 题目提供的 GitHub 内容	5
3.1 server/: 本地转发服务	6
3.2 docs/: 接口和交付说明	6
3.3 web/: 前端空壳	6
3.4 从讲义中理解到的原理	6
4 实现内容与关键原理	7
4.1 请求封装与登录	7
4.2 房间与弹幕接入	7
4.3 参与池与公平抽奖	8
4.4 状态管理与持久化	8
4.5 多房间与主题	8
5 遇到的问题与解决	9
6 总结与体会	9
7 补充技术实现	9
7.1 主题色自定义取色	9
7.2 多房间合并池	10
7.3 名单与设置页布局调整	10
7.4 自定义背景与主题性能	10
7.5 登录态封装与过期处理	11
7.6 转发服务与题目仓库一致	11

目录	2
8 展示：启动与运行流程	11
8.1 端口检查	11
8.2 自动打开浏览器	11

1 实习概述

本次实习的任务是完成一个 B 站直播弹幕抽奖网页。用户登录后可以添加直播间，页面持续接收该直播间的弹幕，按照关键词把观众加入参与池，最后随机抽出若干名中奖者。完整链路为：



题目要求只使用前端三件套，不使用框架，也不使用构建工具。因此整个项目由三部分组成：

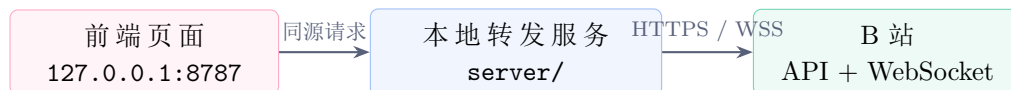
- **前端页面：**界面、交互、请求、弹幕解析、参与池、抽奖和本地存储；
- **题目提供的本地转发服务：**负责静态托管、B 站接口反向代理和弹幕 WebSocket 中转；
- **B 站服务：**提供账号、直播间、弹幕配置等接口，以及持续推送弹幕的 WebSocket 服务。

项目最终完成了 P0、P1、P2 的主要功能，包括 Cookie 登录、扫码登录、房间号解析、弹幕解帧解压、关键词抽奖、多套配置、历史记录、主题切换、断线重连和多房间合并。

2 整体框架搭建

2.1 总体架构

页面不能直接访问 B 站接口：浏览器有同源策略，并且禁止脚本设置 Cookie 请求头。所以页面只跟本机的转发服务通信，由转发服务把请求发送给 B 站。



页面使用相对路径访问：

本机路径	转发到	用途
/api/...	api.bilibili.com	账号、WBI 密钥
/live/...	api.live.bilibili.com	房间信息、弹幕配置
/passport/...	passport.bilibili.com	扫码登录
/ws	弹幕 WebSocket	二进制字节原样中转

2.2 前端模块划分

前端没有使用框架，但按职责拆成了多个 ES Module。这样做的目的是让协议、请求、状态和界面互不引用，出问题能快速定位。

模块	职责
main.js	入口：启动、视图切换、主题、语言
api.js	请求封装、B 站接口、房间号解析
wbi.js / md5.js	WBI 签名与 MD5
danmaku.js	弹幕协议：组包、解帧、解压、解析、心跳、重连
pool.js	参与池、关键词过滤、抽奖算法
rooms.js	多房间连接管理
store.js	单一状态源与 localStorage 持久化
ui/	各视图控制器、模态框、toast、账号面板
i18n.js / qrcode.js / exporter.js	中英双语、二维码、Markdown 导出

2.3 运行方式

项目使用题目提供的 Go 转发服务：

```
1 cd server
2 go run . -addr 127.0.0.1:8787 -web ../web
```

然后打开 <http://127.0.0.1:8787>。不能直接双击 index.html，因为 file:// 下 ES Module 会被浏览器拦截，而且没有服务可以转发 /api、/live、/passport 和 /ws。

框架搭建的关键点

1. 页面只获取同源数据，跨域和 Cookie 问题交给本地服务处理；
2. 协议、请求、状态、渲染分层，避免把网络逻辑和界面逻辑混在一起；
3. 所有状态集中到 store.js，界面只读状态、只通过 store 方法修改状态。

2.4 HTML 与 CSS 的基本构成

整个页面由 HTML 描述结构，由 CSS 描述样式，再由 JavaScript 负责交互和数据。这三部分职责分开，页面才不会写成一团。

2.4.1 HTML 的基本构成

一个 HTML 文档的基本骨架是：

```
1 <!doctype html>
2 <html lang="zh-CN">
3   <head>
4     <meta charset="UTF-8" />
5     <meta name="viewport" content="width=device-width, initial-scale=1.0" />
6     <title>GACHAGO</title>
7     <link rel="stylesheet" href="../css/base.css" />
8   </head>
9   <body>
10    <header class="topbar">...</header>
11    <main>
12      <section id="view-draw">...</section>
13      <section id="view-settings" hidden>...</section>
```

```
14     </main>
15     <script type="module" src="./js/main.js"></script>
16 </body>
17 </html>
```

主要概念包括：

- 标签和元素：<h2>、<p>、<button> 等描述内容；
- 属性：class、id、type、value 等补充信息；
- 语义化标签：<header>、<main>、<section>、<article> 让结构更清楚；
- 表单：<label> + <input>、<select>、<button>;
- 自定义数据：data-*, 例如 data-room-id;
- 可访问性：alt、aria-label、role、aria-selected。

2.4.2 CSS 的基本构成

CSS 规则由“选择器 + 声明块”组成：

```
1 .card {
2   padding: 16px;
3   border-radius: 12px;
4   background: #ffffff;
5 }
```

主要概念包括：

- 选择器：元素、类、ID、属性、后代、伪类；
- 盒模型：content、padding、border、margin，项目统一用 box-sizing: border-box;
- 布局：display: block / inline / flex / grid;
- 层叠和优先级：越具体的选择器越优先，后写的覆盖先写的；
- CSS 变量：例如 --accent、--surface、--text;
- 响应式：用 @media (max-width: ...) 在窄屏改变列数和间距。

项目里 CSS 分成三个文件：base.css 放 reset、变量和主题，components.css 放按钮、卡片、输入框等组件，views.css 放抽奖页和设置页的具体布局。这样修改样式时不会互相影响。

3 题目提供的 GitHub 内容

题目给出的仓库是：

<https://github.com/aDarkMaker/BiliLuckyDraw/tree/BingyanFE-Practice>

仓库里和我直接相关的部分主要有三类。

3.1 server/: 本地转发服务

这是一个只负责转发的 Go 服务，不包含抽奖业务逻辑。它做三件事：

1. **静态托管**：把 web/ 目录作为静态文件目录，浏览器从 127.0.0.1:8787 打开页面；
2. **反向代理**：把 /api、/live、/passport 开头的请求分别转发到 B 站的三个域名；
3. **WebSocket 中转**：把 /ws 的二进制字节原样转发给 B 站弹幕节点，服务本身不解析弹幕。

转发服务解决了两个浏览器限制：

- **同源策略**：页面和转发服务同源，所以浏览器不会拦截；
- **Cookie 头限制**：浏览器禁止脚本设置 Cookie 头，所以页面用 X-Cookie 传递登录态，服务端再把它转换成真正的 Cookie 头。

3.2 docs/: 接口和交付说明

文件	内容
docs/api.md	本地路由、Cookie 约定、B 站接口、弹幕包头
docs/features.md	P0 / P1 / P2 功能清单
docs/delivery.md	交付要求：能跑的仓库、README、演示、已知问题

3.3 web/: 前端空壳

题目给的前端只有最基础的页面骨架。我的实现是在这个空壳上继续写页面、样式、请求封装、弹幕协议、参与池、抽奖、状态管理和界面控制器。

题目仓库和我实现的关系

server/ 保持题目原样，负责转发；web/ 是我实际完成的部分。两者通过 /api、/live、/passport、/ws 这几个约定路径连接。

3.4 从讲义中理解到的原理

结合题目文档和讲义，我重点理解了下面几条：

- 浏览器同源策略和 CORS 限制的是“跨域读取”，不是请求能不能发出去；
- B 站接口统一返回 `\{code, message, data\}`，只有 `code === 0` 才算业务成功；
- 弹幕 WebSocket 传的是二进制包，不是普通 JSON；
- 参与池必须按 uid 去重，昵称不是唯一标识；
- 抽奖必须使用公平洗牌，不能用 `sort(() => Math.random() - 0.5)`。

4 实现内容与关键原理

4.1 请求封装与登录

所有网络请求都走 `api.js` 的 `request()`。它负责拼接 `query`、带上 `X-Cookie`、设置超时、解析 JSON，并把 HTTP 错误和业务 `code !== 0` 统一抛成异常。

```
1 const res = await fetch(url, {
2   headers: {
3     Accept: 'application/json, text/plain, */*',
4     ...(cookie ? { 'X-Cookie': cookie } : {}),
5   },
6   signal: controller.signal,
7 });
8 const json = await res.json();
9 if (json.code !== 0) throw new Error(json.message);
```

登录支持两种方式：

- **Cookie 登录：**兼容整段 `document.cookie`、`Cookie`：头和单独的 `SESSDATA` 值；
- **扫码登录：**申请二维码后每 2 秒轮询，处理 0（成功）、86090（已扫待确认）、86038（过期）三种状态。

getDanmuInfo 的 WBI 签名

这个接口不带 `w\_rid` 和 `wts` 会返回 -352 风控失败。签名步骤是：

1. 从 `/api/x/web-interface/nav` 取 `img\_url` 和 `sub\_url`，提取文件名作为 `img\_key` 和 `sub\_key`；
2. 把两个 key 拼接后按固定的 64 项置换表重排，取前 32 位作为 `mixin\_key`；
3. 参数按键名排序、值去掉 `!()*、URL` 编码，接上 `wts`；
4. 计算 `w\_rid = md5(query + mixin\_key)`。

签名串必须原样发送，不能再让 URL 编码器重新编码，否则签名会对不上。

4.2 房间与弹幕接入

房间号解析支持纯数字、直播间链接、`/blanc/` 路径和 `query` 参数。短号不是真实房间号，必须用 `get\_info` 返回的 `data.room\_id`，必要时再调用 `mobileRoomInit`。

拿到真实房间号后，调用 `getDanmuInfo` 获得 `token` 和 `host\_list`，然后连接 `WebSocket`。弹幕协议是所有功能里最关键的部分。

偏移	类型	字段
0	int32	整包长度（含 16 字节头）
4	int16	头长度，固定 16
6	int16	协议版本：0/1 JSON，2 deflate，3 brotli
8	int32	operation：2 心跳 / 3 心跳回 / 5 消息 / 7 鉴权 / 8 鉴权成功
12	int32	sequence
16	bytes	包体

处理流程是：

1. 连接成功后先发 `op = 7` 的鉴权包；
2. 用 `splitPackets()` 按 16 字节包头切包；
3. `ver = 2` 时用 `DecompressionStream('deflate')` 解压，失败再试 `'deflate-raw'`；
4. 解压出来的仍然是“包头 + 包体”，所以递归回到切包函数；
5. 只处理 `cmd === 'DANMU\_MSG'`：`info[1]` 是文本，`info[2][0]` 是 uid，`info[2][1]` 是昵称；
6. 每 30 秒发一次 `op = 2` 心跳包。

为什么解压之后要递归

服务器压缩的不是“一条消息”，而是一整段完整的包序列。解压后可能还是多个包，甚至还有压缩包。所以必须递归解包，直到全部变成 JSON。项目里加了深度上限，防止异常数据导致无限递归。

4.3 参与池与公平抽奖

参与池使用 `Map<uid, entry>`：

- 关键词为空时所有人都参与；
- 多个关键词之间是“或”的关系；
- 同一个人发多条弹幕只算一个人，重复发言只增加 `count`。

抽奖使用部分 Fisher-Yates：只在副本上做前 `N` 次交换，保证结果不重复。随机数优先使用 `crypto.getRandomValues`，并用拒绝采样去掉取模偏差。点“开始抽奖”时会清空上一轮参与池，避免第二轮抽到历史总人数。

为什么不能用 `sort(random)` 洗牌

`sort(() => Math.random() - 0.5)` 的比较函数不满足传递性，不同元素被换到各位置的概率不均等，抽奖结果有偏。Fisher-Yates 从后往前，每次和前面随机位置交换，才是公平的。

4.4 状态管理与持久化

`store.js` 是唯一状态源，保存语言、主题、Cookie、用户信息、多套配置和抽奖历史。读取本地存储时一定用 `try/catch`，并把旧数据和默认值合并，以后新增字段不会让旧数据把页面弄崩。配置文件、历史、主题、语言会跨刷新保留；连接状态、参与池和弹幕日志不写入存储。

4.5 多房间与主题

多房间使用一个房间一条 WebSocket 的方式，由 `RoomManager` 管理。所有房间共用一个参与池，每条记录保存 `rooms: Set<roomId>`，另建 `Map<roomId, Set<uid>>` 反查索引。断开某个房间时只移除该房间的贡献，同时在别的房间发过弹幕的人会留在池子里。

主题使用 CSS 变量：切换时只改 `<html data-theme>` 和主题色变量，不需要给每个元素写两套样式。主题色支持 4 套预设和自定义取色；自定义颜色会自动计算深色版本和深浅模式用的半透明色。切换瞬间会临时关闭过渡，避免整页重绘掉帧。

5 遇到的问题与解决

问题	解决思路
直接请求 B 站被 CORS 拦截	改用 <code>/api</code> 、 <code>/live</code> 、 <code>/passport</code> 相对路径，走本地转发服务
Cookie 无法设置	页面用 <code>X-Cookie</code> ，服务端再转成 <code>Cookie</code> 头
<code>getDanmuInfo</code> 返回 -352	实现 WBI 签名，并在失败时退到公共弹幕节点
解压后还是二进制	压缩的是包序列，解压后递归回切包函数
连接状态和界面不一致	先拆后建、界面镜像 socket 状态、失败分支必须收尾
头像裂图	<code>referrerpolicy="no-referrer"</code> + 昵称首字渐变头像回退
主题切换掉帧	切换瞬间关闭过渡，减少大面积毛玻璃和热路径颜色计算

6 总结与体会

这次实习让我把一个前端页面从“能显示”推进到了“能真正处理实时数据”。最大的收获不是写页面，而是理解了浏览器、转发服务、B 站接口和 WebSocket 之间的关系：

- 页面不能直接跨域访问 B 站，必须通过本地转发服务；
- B 站接口有 HTTP 状态码和业务 `code` 两层结果，必须分开处理；
- 弹幕是二进制协议，要按包头切包、按版本解压、递归处理；
- 参与池必须按 `uid` 去重，抽奖必须使用公平洗牌；
- 状态必须有单一数据源，界面不能偷偷保存一份自己的状态；
- 网络程序必须考虑断线、超时、重连和用户随时停止操作。

目前仍有几处可以继续完善：`b23.tv` 短链尚未展开；Cookie 存在 `localStorage` 只适合本地练习；演示部分还可以补一段真实直播间的完整录屏。后续如果继续迭代，我会优先补充自动化测试、短链解析和更完善的错误提示。

7 补充技术实现

在核心链路完成之后，我又根据实际使用体验做了几处补充和调整。

7.1 主题色自定义取色

主题色最初只有 4 套预设。后来加入了原生取色器：预设色块旁边有一个彩虹圆点，点击后可以选择任意颜色。自定义颜色会立即应用到按钮、选中态和渐变等位置，并保存到 `localStorage`，刷新后仍然保留。

实现上没有写死第二套主题，而是把自定义颜色转换成一组 CSS 变量：

- `--accent`: 用户选择的主色;
- `--accent-strong`: 加深后的颜色, 用于渐变和强调;
- `--accent-soft-light` / `--accent-soft-dark`: 浅色和深色模式下的柔和底色;
- `--accent-tint-light` / `--accent-tint-dark`: 半透明着色;
- `--accent-ink`: 根据颜色的亮度自动选择白色或深色文字。

对于自定义颜色, 文字和徽章的对比色不写死: `web/js/main.js` 的 `luminance()` 计算颜色的相对亮度, 亮度大于 0.6 时使用深色 `##10131a`, 否则使用白色 `##ffffff`, 结果存入 `--accent-ink`。

这样预设主题和自定义颜色可以无缝切换。选中的色块显示黑色圆环和白色对勾, 自定义取色也走同一套选中状态。

7.2 多房间合并池

多房间是后期补充的重点功能。B 站没有“一条连接听多个房间”的能力, 所以实现方式是一个房间一条 `WebSocket`, 由 `RoomManager` 统一管理。数据层把多个房间的弹幕放进同一个参与池:

- `entries: Map<uid, entry>` 是主索引, 同一个人只算一条记录;
- 每条记录保存 `rooms: Set<roomId>`, 表示这个人出现在哪些房间;
- `byRoom: Map<roomId, Set<uid>>` 是反查索引, 用来统计和移除单个房间;
- 断开某个房间时只移除该房间的贡献; 如果这个人也在别的房间发过弹幕, 他仍然留在池子里;
- 按房间抽奖时使用 `taken` 集合, 保证同一个人不会在两个房间重复中奖。

这套结构解决了“合并成一个池子”和“房间之间互不干扰”两个问题。

7.3 名单与设置页布局调整

中奖名单原来使用两列窄卡片, 长昵称会被挤成竖排。后来改成一行一个中奖者的横向布局: 序号、完整昵称、标签和 `uid` 按顺序排列, 昵称占主要宽度, 长昵称可以正常换行, 不会再出现一个字一行的现象。

设置页也做了布局调整: 左列是“账号 → 抽奖历史”, 中间是“配置方案”, 右侧是“外观”。这样查看历史时不需要往下滚很久, 同时保留了原来的卡片样式和网格规则。

7.4 自定义背景与主题性能

自定义背景图使用 `canvas` 先压缩, 再转成 `JPEG` 保存到 `localStorage`, 避免手机照片直接超过存储上限。后期把自定义背景下的毛玻璃效果调弱了:

- 顶栏模糊从 18px 降到 10px, 饱和度从 1.6 降到 1.25;
- 壁纸暗色遮罩和浅色 / 深色蒙层的不透明度降低, 背景图更清楚;
- 主题切换瞬间临时关闭过渡, 避免整页重绘造成掉帧。

7.5 登录态封装与过期处理

登录 Cookie 不再作为裸字符串永久保存，而是封装成带时间信息的会话对象：

- 保存 `savedAt`、`checkedAt`、`expiresAt` 三个时间戳；
- 页面启动和长时间空闲后，会自动调用 `/api/x/space/myinfo` 校验会话；
- 任何接口返回 `-101` 未登录时，广播会话过期事件，清空 Cookie 和用户信息，提示重新登录；
- `getDanmuInfo` 返回的 `token` 不写入本地存储，每次连接弹幕前按需重新获取。

这样处理的目的是把 Cookie 从“只存不查”的原始字符串，变成有验证、有清理、有过期提示的会话状态，避免服务端 Cookie 已经失效但页面仍显示已登录的情况。

7.6 转发服务与题目仓库一致

最终提交的 `server/` 保持题目原版：

- `server/main.go`、`go.mod`、`go.sum` 与题目 GitHub 仓库的 `BingyanFE-Practice` 分支一致；
- 转发服务只负责静态托管、B 站接口反向代理和 WebSocket 二进制中转；
- 解帧、解压、参与池、抽奖和界面逻辑全部在 `web/` 前端完成。

8 展示：启动与运行流程

展示时可以直接双击项目根目录的 `start.bat`。它会自动完成端口检查、启动服务和打开页面三步。

8.1 端口检查

脚本先检查 8787 端口：

- 如果 8787 已经在运行 GACHAGO，脚本不再启动第二个服务，直接打开页面；
- 如果 8787 被别的程序占用，脚本自动改用 8788，避免两个项目冲突；
- 如果 8787 空着，就正常在 8787 启动。

判断“是不是 GACHAGO”的方式是请求 `/js/store.js`，检查返回内容里有没有 `gachago.state.v1` 这个存储键名。

8.2 自动打开浏览器

脚本用一条隐藏的 PowerShell 命令完成浏览器打开：

```
1 start "" powershell -NoProfile -WindowStyle Hidden -Command ^
2 "for($i=0;$i -lt 40;$i++){ try { $c = New-Object Net.Sockets.TcpClient; ^
3 $c.Connect('127.0.0.1',8787); $c.Close(); break } ^
4 catch { Start-Sleep -Milliseconds 300 } }; Start-Process 'http://127.0.0.1:8787'"
```

它的逻辑是：

1. `start` 在后台启动一个隐藏的 PowerShell 进程，这样批处理可以继续启动 Go 服务；
2. PowerShell 每隔 300ms 尝试连接一次 127.0.0.1:8787，最多试 40 次（约 12 秒）；
3. 端口通了以后执行 `Start-Process 'http://127.0.0.1:8787'`，由 Windows 调用默认浏览器打开页面；
4. 同时批处理在前台执行 `go run . -addr 127.0.0.1:8787 -web ../web`，这个窗口就是服务本身，关闭窗口服务就停止。

如果浏览器没有自动打开，也可以手动访问 `http://127.0.0.1:8787`。